Efficacy of a Gaussian Mixture Model based Speaker Recognition in Machine Learning Framework

Final report on the project work submitted for fulfillment of summer internship by

Adhiraj Banerjee

3rd Year B.Tech, Electronics & Comm. Engg.

Indian Institute of Engineering, Science and Technology (IIEST), Shibpur

Under the Supervision of

Dr. Siva Ram Krishna Vadali

Senior Principal Scientist

Robotics & Automation Division

CSIR-Central Mechanical Engineering Research Institute Duragpur

August 2021

Contents

1 Introduction **4**

2 "Machine Learning" for Speaker Recognition **4**

2.1 Gaussian Mixture Model based SR 5

2.1.1 The E-M Algorithm 6

2.2 Feature Extraction in SR: Mel-Frequency Cepstrum Coefficients(MFCC) 9

3 Observations in accuracy of GMM in SR for Speech samples with noise **15**

4 Results and Inference **17**

5 Conclusion on Work Done **18**

List of Figures

1 Framing of voice signal 11

2 Relationship between Mel-scale and Hz-scale 13

3 Triangular Mel-Filter Bank 14

4 Conversion of Spectrum to Cepstrum 14

5 Audio Signals with & without noise 17

1 Introduction

Speaker Recognition (SR) has numerous applications such as voice biometric, forensics, traitor finding, voice-mail and tele-banking. Conceptually, SR determines the presence of the active speaker among a set of registered speakers. In view of the applicability of Machine Learning framework to a wide variety of classification problems, the present work studies the popular Gaussian Mixture Model (GMM) based ML (GMM-ML) approach as a potential solution for speaker recognition. Voice samples of ten different speakers are recorded and the performance of GMM-ML for SR is analyzed. Further, performance of GMM-ML is also studied for SR when the speech signal is overlaid with additive noise. It is observed that GMM-ML based SR is fairly accurate in distinguishing the correct speaker from the 10 registered speakers.

2 "Machine Learning" for Speaker Recognition

Machine Learning (ML) is a mathematical framework which helps systems to learn and train a model with (preferably) large data sets and subsequently predict the category into which a test data may be classified to. The present work studies the efficacy of a ML based classification algorithm for Speaker Recognition. Going by principles of ML, a model for Speaker Recognition is trained such that the model identifies unique patterns (referred as features)

from a set of different voice samples (of registered speakers) and distinguish the actual speaker voice sample from the rest. In the present work, a familiar ML algorithm, Gaussian Mixture Model (GMM) based classification, is chosen and its efficacy in accurate speaker recognition is studied.

2.1 Gaussian Mixture Model based SR

It is known that the Gaussian Mixture Model (GMM) is a powerful model used to solve clustering problems. It is a probabilistic approach by which it approximates the probability distribution of a MXK length data (M is the number of K dimensional data vectors) belonging to a class λ (of L classes) as a linear combination of N Gaussian distributions or clusters with mean vector $\boldsymbol{\mu}$=$\{\mu_i\}_{i=1,2,...,N}$ (i.e. order $KX1$), variance-covariance matrix $\boldsymbol{\Sigma}$ (i.e. order KXK) and $\boldsymbol{\omega}$=$\{\omega_i\}_{i=1,2,...,N}$ (i.e. order MXN) as the weights in the linear combination. In *Speaker Recognition*, features of speaker's voice sample are extracted into a feature vector and used as data points for processing to identify the registered speaker. Since the feature vector is *multi-dimensional*, the probability density function (PDF) of particular class of speaker (λ) can be expressed as a linear combination of N multivariate Gaussian PDFs as:

$$P(\boldsymbol{X}|\lambda) = \sum_{i=1}^{N} \omega_i \cdot P(\boldsymbol{X}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \tag{1}$$

and,

$$P(\boldsymbol{X}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) = \prod_{n=1}^{M} P(\boldsymbol{x}_n|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \tag{2}$$

where, M denotes the number of Gaussian random variables used for N clusters formed from the input vector $\boldsymbol{X}$; λ, as mentioned earlier, is the class to which a speaker is associated with; $\boldsymbol{X}$ represents the N dimensional training data; and $P(\boldsymbol{x}_n|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$ is the multivariate Gaussian PDF given by,

$$P(\boldsymbol{x}_n|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) = \left((2\pi)^N \cdot |\boldsymbol{\Sigma}_i|\right)^{-0.5} \cdot exp\left(-0.5(\boldsymbol{x}_n - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_i)\right). \quad (3)$$

In the formulation in Equation 1, the probability, also referred as the likelihood, computed from the input data points in vector X is assigned to a particular class corresponding to the speaker's GMM. Herein, the mean vectors ($\boldsymbol{\mu}_i$), variance-covariance matrices ($\boldsymbol{\Sigma}_i$) and weights of each of the gaussian components ($\boldsymbol{\omega}_i$) is updated iteratively till the likelihood for the data points (i.e. feature vectors) is maximized, thereby creaing a trained model with the extracted features. The maximized likelihood will thus be assigned to the class corresponding to the speaker, in turn creating the the classified GMM for the speaker.

2.1.1 The E-M Algorithm

The *E(Expectation)-M(Maximization)* Algorithm is an iterative statistical algorithm which is used to train Gaussian Mixture Models by updating the unknown parameters($\boldsymbol{\mu}_i$,$\boldsymbol{\Sigma}_i$ & $\boldsymbol{\omega}_i$) till convergence is achieved. In GMMs, the mean vector, the covariance matrix and the weights of each Gaussian com-

ponent, for obtaining the maximum log-likelihood of datapoints is unknown, which brings the significance of this algorithm. This algorithm is executed in this fashion:

- Initialization: At first, arbitary values are taken for initializing the mean vector($\boldsymbol{\mu}_i$), covariance matrix($\boldsymbol{\Sigma}_i$) and weights($\boldsymbol{\omega}_i$) for each of the N Gaussian Distributions or clusters. From these initialized values, the initial Log-likelihood is calculated.

- Estimation: Each of the weights can be considered as a prior probability for each component among all the N Gaussian Clusters. In this step, for every data point or vector(K-Dimensional $\boldsymbol{x}$=$\{x_n\}_{n=1,2,...,M}$), their responsibilities($\boldsymbol{\gamma}_i(\boldsymbol{x})$ for every gaussian distribution is determined. Responsibilities of every data point can be calculated as their corresponding *Posterior Probabilities* from the previously initialized parameters of each Gaussian component using **Bayes' Theorem**:

$$\boldsymbol{\gamma}_i(\boldsymbol{x}) = \frac{\boldsymbol{\omega}_i \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{j=1}^{N} \boldsymbol{\omega}_j \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \; where, \boldsymbol{\omega}_i = \frac{\boldsymbol{M}_i}{\boldsymbol{M}} \tag{4}$$

  Here, $\boldsymbol{M}_i$ is the effective number of data points assigned to the gaussian cluster $\boldsymbol{i}$ and $\boldsymbol{M}$ is the total number of datapoints for each speaker.

- Maximization: In this step, the parameters of each of the N Gaussian Clusters are re-estimated using the current Responsibilities of each data vector $\boldsymbol{x}$=$\{x_n\}_{n=1,2,...,M}$ for a particular Gaussian component. The

mathematical form for each of the paramaters of a Gaussian Cluster is given below:

1. **Mean Vector($\boldsymbol{\mu}_i$)**:

$$\boldsymbol{\mu}_i = \frac{\sum_{n=1}^{M} \boldsymbol{\gamma}_i(\boldsymbol{x}_n)\boldsymbol{x}_n}{\sum_{n=1}^{M} \boldsymbol{\gamma}_i(\boldsymbol{x}_n)} \tag{5}$$

2. **Covariance Matrix($\boldsymbol{\Sigma}_i$)**:

$$\boldsymbol{\Sigma}_i = \frac{\sum_{n=1}^{M} \boldsymbol{\gamma}_i(\boldsymbol{x}_n)(\boldsymbol{x}_n - \boldsymbol{\mu}_i)(\boldsymbol{x}_n - \boldsymbol{\mu}_i)^T}{\sum_{n=1}^{M} \boldsymbol{\gamma}_i(\boldsymbol{x}_n)} \tag{6}$$

3. **Weight of each Cluster($\boldsymbol{\omega}_i$)**:

$$\boldsymbol{\omega}_i = \frac{1}{\boldsymbol{M}} \sum_{n=1}^{M} \boldsymbol{\gamma}_i(\boldsymbol{x}_n) \tag{7}$$

- <u>Evaluating the Log-Likelihood</u>: Here, from the estimated parameters of each Gaussian component in the previous step, the Log-Likelihood of the data points belonging to a particular class of speaker $\boldsymbol{\lambda}$ is calculated, as shown below:

$$\ln P(\boldsymbol{X}|\lambda) = \sum_{n=1}^{M} \ln \sum_{i=1}^{N} \boldsymbol{\omega}_i \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \tag{8}$$

If convergence of data points to the $\boldsymbol{N}$ Gaussian Clusters is not fulfilled, then starting from the second step, all the steps need to be repeated again. The criteria for which convergence has occurred in the E-M Algorithm is as follows:

1. If the parameters of the Gaussian components($\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i \,\&\, \boldsymbol{\omega}_i$) does not change in the last successive iterations.

2. If the Log-Likelihood of the data points for a particular speaker remains the same(indicating its maximization) in the last few iterations.

2.2 Feature Extraction in SR: Mel-Frequency Cepstrum Coefficients(MFCC)

The features which are extracted from the voice samples, are called, *MFCC (Mel-Frequency Cepstrum Coefficients).* MFCCs have become a prominent feature to be used for feature extraction from train data, because it gives an idea about the perceived difference between frequencies, especially in higher frequency region. Its importance comes from the fact that, *the human ear can perceive voice signal frequncies non-linearly.* From here, there is a clear understanding that while using Signal-Processing in this case, frequencies needs to be analyzed in the *Mel* scale, rather than the Hertz scale. There is logarithmic(non-linear) relationship between the Mel and Hertz scale, from which frequncies in Mel scale becomes more useful. The extraction of MFCCs can be theoretically elaborated in the steps below:

- **Pre-Emphasis:** At first, Pre-Emphasis(after discretization of the analog voice signal) is done to increase the energy of the signal at higher frequencies, the frequency range where the human ear tends to perceive less in linear scale. Pre-Emphasis is mainly done for transmission cases which brings the significance of prescence of higher frequency components in the voice signal. This process is done by passing the signal through a *First-Order High Pass Filter*. The equation for the signal (x[n]) and its output signal (y[n]) after passing through the filter is below:

$$\boldsymbol{y[n] = x[n] - \alpha x[n-1]} \, \forall \, \boldsymbol{\alpha} \in (0.9, 1) \tag{9}$$

  Thus, the z-transform of the filter will be :

$$\boldsymbol{H(z)} = 1 - \boldsymbol{\alpha z}^{-1} \tag{10}$$

- **Framing:** Due to changes in *Prosody*(features of voice) and *random variations* in the vocal tract, the voice signal is a *non-stationary signal(or, random signal)*. However, within short intervals the voice signal is assumed to be *stationary*, for which this signal is analysed over short time windows. Hence, for analysis of voice features, framing is required, as it divides the whole speech signal into short frames of **N** samples and the adjacent frames are overlapped by **M** samples($N > M$).

- **Windowing:** This process is done to remove discontinuities between

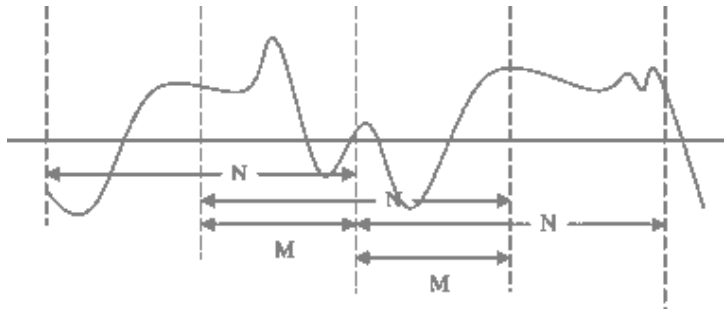

Figure 1: Framing of voice signal

frames of the audio signal. When the whole signal is divided into short frames, discontinuities are formed at the edges of the frame, which are incongruent to the input voice signal. These discontinuities may impact the signal by changing the **Statistical Properties** of the signal. In order to avoid this problem, windowing is executed by multiplying each frame with a window, a smooth function which goes zero at the borders. Generally, the signal frames are multiplied with the *Hamming Window*. Considering the window to be w[n], the equation for windowing is :

$$y[n] = x[n] \cdot w[n] \tag{11}$$

where, $\boldsymbol{x[n]}$ is the Input Signal, $\boldsymbol{y[n]}$ is the Output Signal and for N being the number of samples in each frame,

$$\boldsymbol{w[n]} = 0.54 - 0.46 \cdot \mathbf{cos}\,\frac{\boldsymbol{2\pi n}}{\boldsymbol{N-1}}\,\forall\,\boldsymbol{n} \in [\mathbf{0}, \boldsymbol{N-1}] \tag{12}$$

- **Performing the FFT(Fast Fourier Transform):**We need to exe-

cute the *FFT or N_1-point DFT (Discrete Fourier Transform)* of each frame of the signal to obtain their frequency spectrum from the absolute value of the DFT(time domain to frequency domain). N_1 should be greater than the total sample size of each frame. The spectrum plot is considered from k = 0,.....,$\frac{N_1}{2}$, where $f_{Hz} = k\frac{f_s}{N_1}$, f_s being the sampling frequency and $k\frac{f_s}{N_1}$ is called Resolution. The formula for the FFT can be expressed as:

$$\boldsymbol{X[k] = \sum_{n=0}^{N_1-1} x[n] \cdot \exp \frac{-j2\pi nk}{N_1} \forall k = 0, 1, ..., \frac{N_1}{2}} \tag{13}$$

where, $\boldsymbol{x[n]}$ is the input signal(or, short frame) and $\boldsymbol{X[k]}$ is the FFT of the input x[n]. Here, $\boldsymbol{|X[k]|}$ gives the frequency spectrum of the frames of the input voice signal.

- **<u>Mel-Filter Bank Processing:</u>** The frequency spectrum is then passed through a Mel-filter bank, which is a bank of triangular filters plotted against frequencies in Mel Scale. The relationship between frequencies in Hz scale and frequencies in Mel-scale can be expressed as:

$$\boldsymbol{f_{Mel}} = 2595 \log (1 + \frac{\boldsymbol{f_{Hz}}}{700}) \tag{14}$$

The filter output we get after passing the frequency spectrum into Mel-filter bank gives the Mel-Frequency Spectrum. The equation for

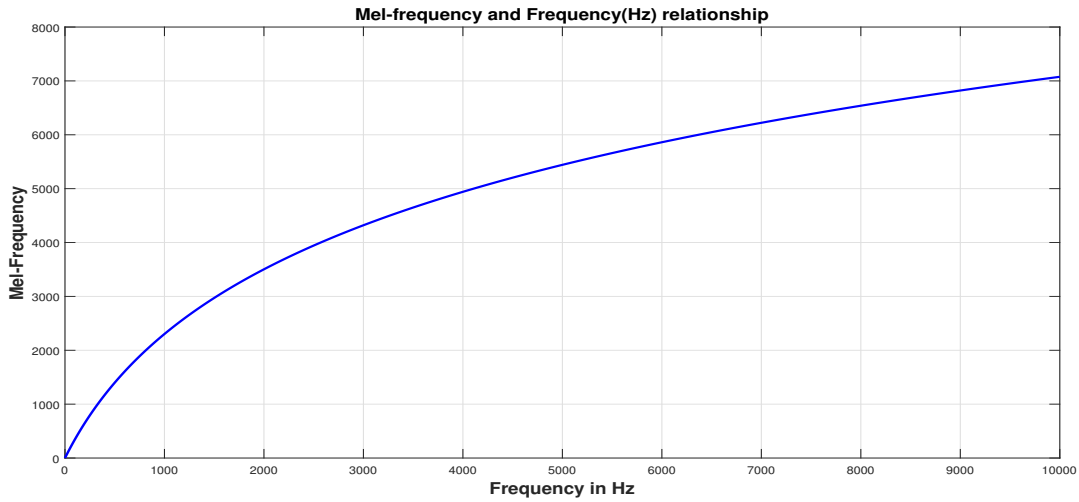

Figure 2: Relationship between Mel-scale and Hz-scale

the filter output $\tilde{S}(l)$ is given below:

$$\boldsymbol{\tilde{S}(l)} = \sum_{k=0}^{N_1/2} \boldsymbol{S(k)} \cdot \boldsymbol{M_l(k)} \, \forall \, \boldsymbol{l} = 0, 1, ..., \boldsymbol{L} - 1 \tag{15}$$

where, $\boldsymbol{S(k)} = |\boldsymbol{X}[\boldsymbol{k}]|$ is the Frequency Spectrum for the input $\boldsymbol{x}[\boldsymbol{n}]$ and $\boldsymbol{L}$ is the number of Triangular Mel-filters used for filtering the whole spectrum to obtain the Mel-frequency spectrum over the whole range of k. The spectrum obtained from filtering operation renders non-linear frequency spectrum at the higher frequencies.The Mel-filter bank usage is important as Voice signal generally follows frequencies in non-linear form and thus the human ear can perceive frequencies being non-linear.

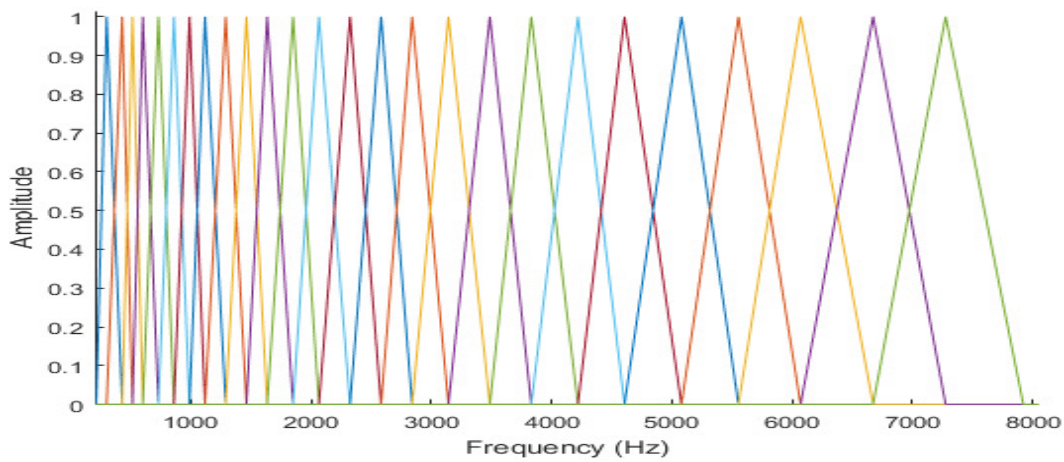

Figure 3: Triangular Mel-Filter Bank

- **Conversion of Mel- Spectrum to Mel-Cepstrum:** The logarithm of the Mel-frequency spectrum is taken to obtain Mel-Frequency Cepstrum.

Figure 4: Conversion of Spectrum to Cepstrum

- **Obtaining the MFCCs:** The Cepstrum obtained in previous step, is operated upon by taking its *Discrete Cosine Transform(DCT)* which provides us MFCCs as the final result. The equation for the performed

DCT is as follows:

$$\boldsymbol{c(i)} = \sqrt{\frac{2}{L}} \sum_{m=1}^{L} \mathbf{log}\,(\boldsymbol{\tilde{S}(m)}) \cdot \mathbf{cos}\,(\frac{\boldsymbol{\pi i}}{\boldsymbol{L}}(\boldsymbol{m} - \mathbf{0.5}))\, \forall\, \boldsymbol{c} = 0, 1, ..., \boldsymbol{C} - 1 \tag{16}$$

where, $\boldsymbol{C}$ is number of MFCCs desired, $\mathbf{log}\,(\boldsymbol{\tilde{S}(m)})$ is the Mel-Cepstrum obtained and $\boldsymbol{L}$ is the number of triangular mel-filters used for obtaining the Mel-Spectrum.

3 Observations in accuracy of GMM in SR for Speech samples with noise

In the above sections, it has been discussed that how from the speech samples provided, the features are extraced from the training files using which the Gaussian Mixture Models(GMM) are trained. It has been observed that, for the speech samples which had been recorded and stored for training and testing purposes have given good results(100% approx.).

In practical situations, noise from the surrounding areas are of random nature and may distort the statistical properties of the voice signal. Hence, in order to test the efficiency of GMM for speaker recognition, for a particular class of speaker, GMMs have been trained after applying additional Gaussian Noises with different variances to the audio extracted from the training files, and the accuracy had been tested for a case of noise of particular variance. The Noise Variance versus Accuracy table is given below:

Variance of Noise	Accuracy(in %)
10^{-10}	100
10^{-8}	92.59
10^{-6}	81.48
10^{-5}	59.25
10^{-4}	37.03
10^{-2}	22.22
1	7.407

The above table gives an idea about the efficiency of GMM in Speaker Recognition system. It is acknowledged that for adding noises with variance above 10^{-5} the accuracy of detection goes below 50% which can be explained on the basis of the **Signal-to-Noise Ratio**(i.e., SNR). The approximate power for each of the audio samples used for training phase was observed in python and minimum of them was seen to be just above 10^{-4}. As a result of this, for some speakers the noise with variances above 10^{-5} could distort the audio samples and the GMMs could not be trained properly. Due to this, speaker recognition for some of the speech samples in testing phase could not function, thus, leading to accuracy decreasing below 50%.

The Gaussian Noise has been added to the audio samples of each speaker by creating noise using *numpy.random.normal*(mean,std_dev,audio) in python. If it is considered that $\boldsymbol{y}$ is the output signal with $\boldsymbol{x}$ as the input signal and $\boldsymbol{n}$ as the additional Gaussian noise, then:

$$\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{n} \tag{17}$$

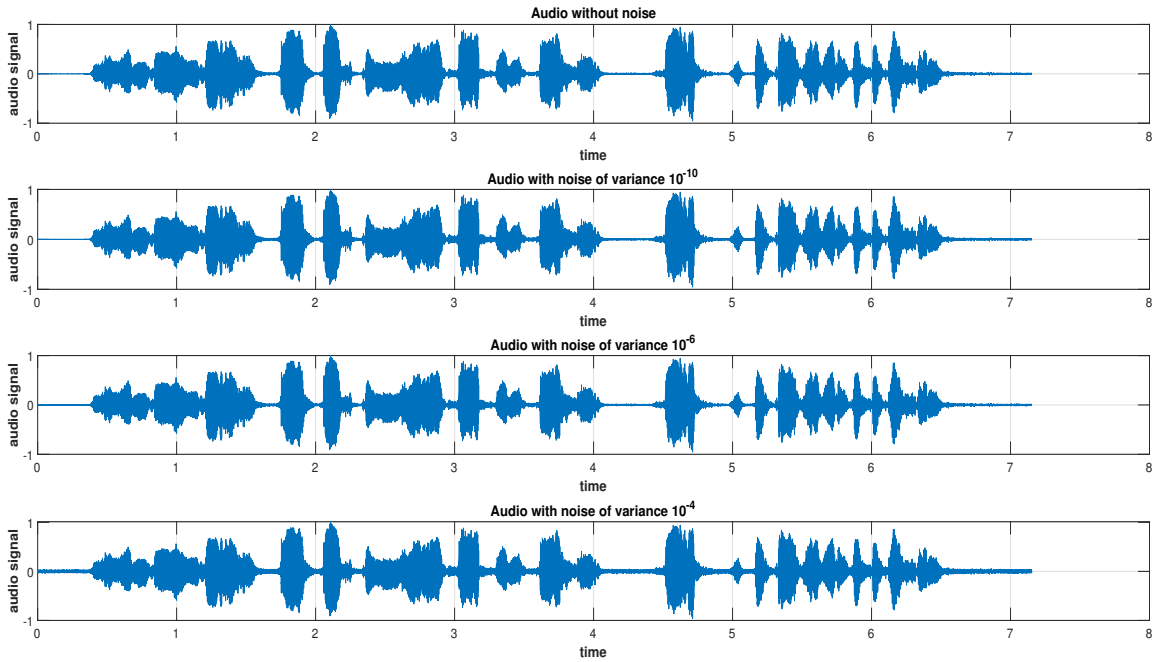

Figure 5: Audio Signals with & without noise

4 Results and Inference

In the present work, GMM based Speaker Recognition is tested for its efficacy over 10 speakers. The entire coding is done in a Python environment. It is observed that GMM based SR is highly accurate (100% approx).Hence, it can be inferred that this approach for accomplishing Speaker Recognition(SR) is highly efficient and can be used in practical applications where SR is important.

5 Conclusion on Work Done

Speech Recognition is a *text-dependent* method, where it highly depends on the language and corpus. On the other hand, Speaker Recognition mainly focusses on raw audio percepts (and from the information derived therein) to identify the uniqueness aspect, if any, among different speakers. *Machine Learning* is a holistic approach which renders *Speaker* Recognition efficient when compared to *Speech* Recognition methods for Speaker idenfitication applications. In this work, performance of GMM-ML is studied for classifying active speaker identification problem, for both clean speech and speech corrupted with additive Gaussian noise. It is found that 100% accuracy may be achieved when tested over 10 different speakers.